TALLER DE ESTRUCTURAS DE DATOS

Pilas, colas y listas enlazadas

Presentado por: Miguel Ángel Tobón Zapata

Programa: Tecnología en Desarrollo de Software

Institución Universitaria Pascual Bravo

Medellín, 2026

Parte 1. Identificación de la estructura adecuada

1. Editor de texto que permite revertir la acción más reciente.

Respuesta: Pila. Esta estructura conserva las operaciones en orden inverso a su ejecución: la última acción almacenada es la primera que se recupera. Este comportamiento se conoce como LIFO (último en entrar, primero en salir).

2. Personas que esperan ser atendidas en una fila.

Respuesta: Cola. La atención respeta el orden de llegada: quien se ubica primero en la fila será atendido primero. Este principio se denomina FIFO (primero en entrar, primero en salir).

3. Lista de canciones que vuelve al inicio cuando termina.

Respuesta: Lista circular. El último nodo mantiene un enlace hacia el primero, de modo que el recorrido puede continuar desde el comienzo sin tener que reconstruir la lista.

4. Registros que se recorren en una dirección y se agregan habitualmente al principio.

Respuesta: Lista enlazada simple. Cada nodo guarda el dato y una referencia al nodo siguiente. Para insertar al inicio, el nuevo nodo apunta a la cabeza actual y luego la cabeza se actualiza para señalar al nuevo elemento.

5. Reproductor que permite avanzar y retroceder entre canciones.

Respuesta: Lista doblemente enlazada. Cada nodo incluye referencias al nodo anterior y al siguiente. Gracias a esos dos enlaces, es posible recorrer la lista hacia adelante o hacia atrás.

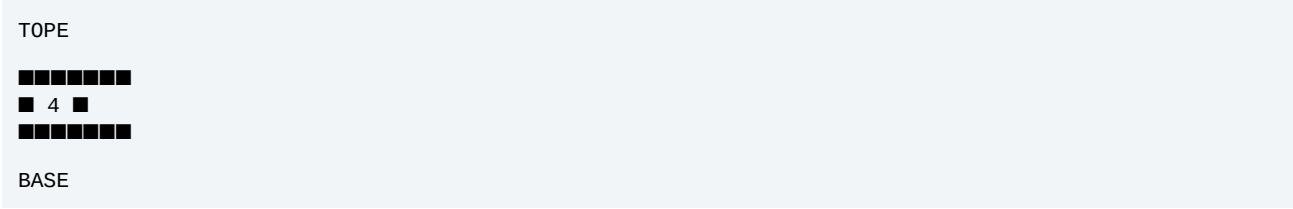
Parte 2. Pilas (LIFO)

2.1. Seguimiento de operaciones

La pila comienza vacía. En la siguiente tabla, los elementos se muestran desde la base hasta el tope. La operación push agrega un dato, pop retira el elemento superior y peek consulta el tope sin eliminarlo.

Operación	Contenido de la pila (base → tope)	Resultado de la operación
push(4)	[4]	Se incorpora el valor 4.
push(9)	[4, 9]	El valor 9 queda en el tope.
push(2)	[4, 9, 2]	Se agrega 2 por encima de 9.
pop()	[4, 9]	Se retira y devuelve el valor 2.
push(7)	[4, 9, 7]	El valor 7 pasa a ser el nuevo tope.
peek()	[4, 9, 7]	Devuelve 7, pero lo mantiene en la pila.
pop()	[4, 9]	Se retira y devuelve el valor 7.
pop()	[4]	Se retira y devuelve el valor 9.

Representación final



Al finalizar las operaciones, el único dato que permanece es 4. Como no hay otros elementos encima, este valor también ocupa la posición del tope.

2.2. Situaciones límite

1. ¿Qué sucede si se ejecuta pop() cuando la pila está vacía?

Antes de retirar un elemento, el programa debe verificar que la pila contenga datos. Si está vacía, debe mostrar un aviso y evitar el acceso a una posición inexistente.

2. ¿Qué debe hacer peek() si no hay elementos en la pila?

Debe comunicar que la pila está vacía. Dependiendo de cómo se haya diseñado el programa, puede devolver un valor especial —por ejemplo, NULO— o presentar un mensaje de error.

3. Pseudocódigo para deshacer una acción:

```
Funcion deshacer(pila)
Si pila está vacía Entonces
Escribir "No hay acciones para deshacer"
Retornar NULO
SiNo
accion <- pop(pila)
Retornar accion
FinSi
FinFuncion
```

2.3. Ejemplo: deshacer acciones

Supongamos que se realizan, en este orden, las acciones escribir, borrar y pegar.

Momento	Contenido (base → tope)	Acción que se deshace
Estado inicial	[escribir, borrar, pegar]	Todavía no se ha deshecho ninguna.
Primer deshacer	[escribir, borrar]	Se revierte pegar, porque fue la acción más reciente.
Segundo deshacer	[escribir]	Se revierte borrar.

La acción pegar se revierte antes que borrar porque una pila trabaja con el principio LIFO: el último elemento guardado es el primero que se retira.

Parte 3. Colas (FIFO)

3.1. Simulación de una fila de atención

En esta tabla, el frente de la cola aparece a la izquierda y el final a la derecha. Encolar agrega una persona al final; desencolar atiende y retira a la persona ubicada al frente.

Operación	Cola resultante (frente → final)	Persona atendida / detalle
encolar("Ana")	Ana	Ana queda al frente y también al final.
encolar("Luis")	Ana → Luis	Luis se ubica detrás de Ana.
encolar("Sol")	Ana → Luis → Sol	Sol queda al final de la fila.
desencolar()	Luis → Sol	Se atiende a Ana.
encolar("Mia")	Luis → Sol → Mia	Mia se agrega al final.
desencolar()	Sol → Mia	Se atiende a Luis.
encolar("Teo")	Sol → Mia → Teo	Teo se ubica al final.

Al terminar, Sol, Mia y Teo continúan esperando en ese orden. Sol ocupa el frente de la cola y Teo se encuentra al final.

3.2. Operaciones y casos especiales

1. Pseudocódigo para atender a la siguiente persona:

```
Funcion atender(cola)
Si cola está vacía Entonces
Escribir "No hay personas esperando"
Retornar NULO
SiNo
persona <- desencolar(cola)
Retornar persona
FinSi
FinFuncion
```

2. ¿Qué pasa si se intenta desencolar una cola vacía?

Primero se comprueba si existen personas en espera. Si no hay ninguna, se informa la situación y la cola permanece sin cambios.

3. ¿Por qué agregar al frente y retirar desde el final no cumple FIFO?

Porque ese procedimiento altera el orden esperado de llegada y atención. Para mantener FIFO, las personas deben ingresar por el final y salir por el frente.

Parte 4. Listas enlazadas simples

4.1. Construcción y modificación de una lista

Una lista enlazada simple está formada por nodos conectados entre sí. Cada nodo apunta al siguiente y el último apunta a NULO.

Paso	Operación realizada	Lista después de la operación	Qué cambió
1	Insertar 18 al final	Cabeza → 18 → NULO	El primer nodo contiene 18.
2	Insertar 7 al comienzo	Cabeza → 7 → 18 → NULO	7 se convierte en la nueva cabeza.
3	Insertar 24 al final	Cabeza → 7 → 18 → 24 → NULO	24 queda como último nodo.
4	Insertar 13 después de 7	Cabeza → 7 → 13 → 18 → 24 → NULO	13 se enlaza entre 7 y 18.
5	Eliminar 18	Cabeza → 7 → 13 → 24 → NULO	13 pasa a enlazarse directamente con 24.

Al quitar el nodo 18, se actualiza el enlace del nodo 13 para que apunte directamente al 24. Como 24 sigue siendo el último, su enlace continúa apuntando a NULO.

4.2. Diseño de operaciones básicas

1. Recorrer la lista e imprimir sus elementos:

```
Funcion recorrerLista(cabeza)
actual <- cabeza
Mientras actual != nulo Hacer
Escribir actual.dato
actual <- actual.siguiente
FinMientras
FinFuncion
```

2. Buscar un valor y determinar si existe:

```
Funcion buscar(cabeza, valor)
actual <- cabeza
Mientras actual != nulo Hacer
```

```
Si actual.dato = valor Entonces
Retornar Verdadero
FinSi
actual <- actual.siguiente
FinMientras
Retornar Falso
FinFuncion
```

3. Insertar un nodo al principio:

```
Funcion insertarComienzo(cabeza, valor)
nuevo <- crearNodo(valor)
nuevo.siguiente <- cabeza
cabeza <- nuevo
Retornar cabeza
FinFuncion
```

4.3. Comprobación de casos límite

Si la lista está vacía, cabeza vale NULO. Al insertar un nuevo nodo, su enlace siguiente queda en NULO y cabeza pasa a apuntar a ese nodo.

Si ya hay elementos, el nuevo nodo apunta al que antes era el primero; después, cabeza se actualiza. Los demás enlaces permanecen intactos.

Cuando se elimina el primer nodo, cabeza debe avanzar al segundo. Si la lista tenía un solo elemento, cabeza queda en NULO.

Situación	Antes	Después
Eliminar el primer nodo	Cabeza → [7] → [13] → [24] → NULO	Cabeza → [13] → [24] → NULO
Insertar al comienzo de una lista vacía	Cabeza → NULO	Cabeza → [nuevo] → NULO

Parte 5. Listas circulares

En una lista circular, el enlace del último nodo vuelve al primero. Por esa razón, el recorrido no termina en NULO como ocurre en una lista simple.

5.1. Recorrido circular

Lista inicial: P1 → P2 → P3 → P1. El recorrido vuelve a P1 después de visitar P3.

Durante la simulación, P2 finaliza después de recibir su segundo turno. La tabla muestra el proceso atendido y cómo continúa el recorrido.

Turno	Proceso atendido	Estado del proceso	Siguiente proceso
1	P1	Continúa activo	P2
2	P2	Continúa activo; recibe su primer turno	P3
3	P3	Continúa activo	P1
4	P1	Continúa activo	P2
5	P2	Finaliza y se retira de la lista	P3
6	P3	Continúa activo	P1
7	P1	Continúa activo	P3

Después de retirar P2, la estructura queda P1 → P3 → P1. El último nodo, P3, conserva su enlace hacia el primero, P1.

Para detener el recorrido de forma segura, se puede guardar una referencia al nodo inicial y finalizar cuando el puntero regrese a ese mismo nodo. Así se evita recorrer la lista indefinidamente.

5.2. Casos especiales

Caso	Cómo se comporta la lista	Actualización necesaria
Lista vacía	No contiene nodos ni enlaces.	La cabeza permanece en NULO.
Un solo nodo	El nodo se enlaza consigo mismo.	Al eliminarlo, la cabeza pasa a NULO.
Eliminar el primer nodo	El segundo nodo pasa a ser el inicial.	La cabeza se actualiza y el último apunta a la nueva cabeza.
Eliminar el último nodo	El nodo anterior se convierte en el último.	Su enlace siguiente apunta al primer nodo.

Ejemplo con un nodo: Antes, Cabeza → [A] y el enlace de A vuelve a A. Después de eliminarlo, Cabeza → NULO.

Ejemplo con varios nodos: Antes, Cabeza → [A] → [B] → [C] y C apunta a A. Al retirar A, B se convierte en cabeza y C debe apuntar a B.

Parte 6. Listas doblemente enlazadas

En una lista doblemente enlazada, cada nodo mantiene dos referencias: una al nodo anterior y otra al siguiente. El enlace anterior del primer nodo y el siguiente del último se establecen en NULO.

6.1. Recorrido en las dos direcciones

Representación: NULO ← [Lunes] ⇄ [Martes] ⇄ [Miércoles] ⇄ [Jueves] → NULO

Dirección del recorrido	Secuencia de nodos	Explicación
Hacia adelante	Lunes → Martes → Miércoles → Jueves	Se sigue el enlace siguiente de cada nodo.
Hacia atrás	Jueves → Miércoles → Martes → Lunes	Se utiliza la referencia al nodo anterior.

6.2. Cambios en la lista

Lista inicial: Lunes ⇄ Martes ⇄ Miércoles ⇄ Jueves.

Paso	Operación	Lista resultante	Referencias que se actualizan
1	Insertar Domingo antes de Lunes	NULO ← Domingo ⇄ Lunes ⇄ Martes ⇄ Miércoles ⇄ Jueves → NULO	Domingo pasa a ser la cabeza; su anterior queda en NULO y Lunes.anterior apunta a Domingo.
2	Insertar Viernes después de Jueves	NULO ← Domingo ⇄ Lunes ⇄ Martes ⇄ Miércoles ⇄ Jueves ⇄ Viernes → NULO	Jueves.siguiente apunta a Viernes y Viernes.anterior apunta a Jueves; Viernes.siguiente queda en NULO.
3	Eliminar Miércoles	NULO ← Domingo ⇄ Lunes ⇄ Martes ⇄ Jueves ⇄ Viernes → NULO	Lunes/Martes y Jueves se reconectan: Martes.siguiente apunta a Jueves y Jueves.anterior apunta a Martes.

6.3. Comparación de los enlaces

La principal ventaja de una lista doble es que se puede avanzar y retroceder sin volver al inicio. Por ejemplo, desde Jueves es posible regresar a Martes utilizando los enlaces anteriores. Como desventaja, cada nodo necesita almacenar dos referencias, lo que consume más memoria y exige actualizar más enlaces al insertar o eliminar elementos.

Parte 7. Análisis y selección de estructuras

Las complejidades de la tabla suponen que las referencias indicadas están disponibles y que actualizar una referencia toma tiempo constante.

Estructura y representación	Operación	Complejidad	Justificación
Pila enlazada con referencia al tope	Insertar en el tope	O(1)	Se crea el nodo y se actualiza la referencia al tope.
Pila enlazada con referencia al tope	Retirar del tope	O(1)	Se retira el nodo superior y el tope pasa al anterior.
Cola enlazada con referencias al frente y al final	Encolar al final	O(1)	El nuevo nodo se enlaza directamente al final.
Cola enlazada con referencias al frente y al final	Desencolar del frente	O(1)	El frente se mueve al siguiente nodo.
Lista simple sin referencia al último nodo	Insertar al comienzo	O(1)	Solo se modifica la referencia cabeza.
Lista simple sin referencia al último nodo	Insertar al final	O(n)	Es necesario recorrer los nodos hasta localizar el último.
Lista doble con referencia a un nodo conocido	Desconectar ese nodo	O(1)	Se enlazan entre sí el nodo anterior y el siguiente.

Preguntas adicionales

1. ¿Por qué una cola que solo conserva la referencia al frente puede tardar más al agregar al final?

Porque el programa debe recorrer los nodos desde el frente hasta localizar el último. Si también se mantiene una referencia al final, se puede acceder directamente a él. Por eso, en el primer caso la operación cuesta O(n), mientras que en el segundo cuesta O(1).

2. ¿Qué beneficio tiene una lista circular cuando se necesita volver al primer elemento?

El último nodo ya está conectado con el primero, así que el recorrido continúa sin tener que regresar manualmente al inicio. Esto resulta útil en listas de reproducción o en sistemas que asignan turnos de manera repetitiva.

3. ¿Qué dato adicional guarda una lista doble y qué ventaja ofrece?

Además del enlace al siguiente nodo, almacena una referencia al anterior. Esto permite recorrer la estructura en ambos sentidos y facilita retirar un nodo conocido al ajustar las referencias de sus vecinos.